

Research Progress Report

Project:	IsoShard Proxy: A Distributed CP-Mode Transaction Coordinator & SQL Optimizer on AWS
Proponent:	Sarthak Mushrif
Reporting Period:	Weeks 14 & 15
Hours Logged:	20 Hours

1. Executive Summary of Progress

During the seventh reporting period, the primary focus was on resolving a critical concurrency race condition within the system's fault-tolerance architecture and executing the initial AWS cloud migration. The Crash Recovery Daemon was successfully upgraded with a Stale-State Timeout (TTL) mechanism to accurately differentiate between actively processing transactions and genuinely orphaned database locks. Following this stabilization, the Dockerized middleware was deployed to AWS EC2 and placed behind an AWS Network Load Balancer (NLB) to efficiently manage incoming high-throughput binary TCP traffic.

2. Detailed Task Breakdown (20 Hours)

- **Stale-State Timeout (TTL) Implementation (6 Hours):** Modified the recovery daemon to calculate the delta between the current system epoch and the DynamoDB WAL timestamp. The daemon now strictly enforces a 30-second TTL threshold. Transactions residing in a PREPARED state for less than 30 seconds are bypassed as healthy, active threads, entirely eliminating the race condition that previously caused premature rollbacks.
- **AWS EC2 & NLB Provisioning (8 Hours):** Provisioned an AWS Auto Scaling Group utilizing `t3.medium` EC2 instances to host the Dockerized proxy. Configured an AWS Network Load Balancer (NLB) to route raw TCP traffic on port 8080 directly to the Netty EventLoops of the underlying proxy instances, ensuring the binary protocol remains intact without HTTP overhead.
- **Distributed JOIN Research & Architecture (6 Hours):** Began drafting the in-memory processing logic required for cross-shard JOIN operations. Designed the preliminary AST visitor extensions to identify when a query spans multiple physical RDS shards, necessitating the proxy to independently fetch, buffer, and merge the row sets before returning the serialized result to the client.

3. Challenges & Roadblocks

- **NLB Health Check Compatibility:** The AWS NLB strictly requires a health check to route traffic, but standard HTTP ping endpoints were incompatible with IsoShard's custom binary protocol. This was resolved by implementing a lightweight, specific 1-byte ping/pong command within the Netty pipeline dedicated exclusively to responding to AWS health checks.
- **Distributed Clock Drift:** Relying on timestamps for the TTL logic introduced the risk of NTP clock drift between the EC2 instances generating the timestamps and the daemon evaluating them. Mitigated this by strictly configuring all EC2 environments to synchronize via the Amazon Time Sync Service.

4. Objectives for Next Reporting Period (Weeks 16 & 17)

- Finalize the programmatic implementation of Cross-Shard JOIN aggregations within the proxy's memory space.

- Conduct high-concurrency TCP load testing against the AWS NLB to measure true network throughput and pinpoint any lingering Netty thread bottlenecks.
- Develop a localized caching layer for frequently accessed, read-heavy queries to further bypass database I/O latency.

Appendix A: Technical Artifact - TTL-Enforced Recovery Logic

The following Java snippet demonstrates the updated reconciliation logic. By implementing a 30-second timestamp delta, the daemon deterministically targets only crashed transactions while ignoring healthy, in-flight processes.

```
1 import software.amazon.awssdk.services.dynamodb.DynamoDbClient;
2 import software.amazon.awssdk.services.dynamodb.model.*;
3 import java.util.Map;
4
5 public class CrashRecoveryDaemon {
6
7     private final DynamoDbClient dynamoClient;
8     private final String tableName = "IsoShard_WAL";
9     private static final long TIMEOUT_THRESHOLD_MS = 30_000; // 30 Seconds
10
11     public void reconcileOrphanedTransactions(ShardManager shardManager) {
12         System.out.println("[Recovery] Scanning WAL for PREPARED transactions ...");
13
14         ScanRequest scanRequest = ScanRequest.builder()
15             .tableName(tableName)
16             .filterExpression("#s = :val")
17             .expressionAttributeNames(Map.of("#s", "State"))
18             .expressionAttributeValues(Map.of(":val", AttributeValue.builder().s("PREPARED").build()))
19             .build();
20
21         ScanResponse response = dynamoClient.scan(scanRequest);
22         long currentTime = System.currentTimeMillis();
23
24         for (Map<String, AttributeValue> item : response.items()) {
25             String txId = item.get("TransactionId").s();
26             long walTimestamp = Long.parseLong(item.get("Timestamp").n());
27
28             // Evaluates the Stale-State Timeout (TTL)
29             if ((currentTime - walTimestamp) > TIMEOUT_THRESHOLD_MS) {
30                 System.out.println("[Recovery] Found dead TxID: " + txId + ". Issuing ROLLBACK.");
31                 shardManager.broadcastRollback(txId);
32                 markAsAbortedInWal(txId);
33             } else {
34                 System.out.println("[Recovery] TxID: " + txId + " is active. Bypassing.");
35             }
36         }
37     }
38
39     private void markAsAbortedInWal(String txId) {
40         // DynamoDB update logic: PREPARED -> ABORTED
41     }
42 }
```

Listing 1: CrashRecoveryDaemon.java (Updated)